EDAN20
Final Examination

Pierre Nugues

October 27, 2025

The examination is worth 200 points: 100 points for the first part and 100 for the second one. The distribution of points is indicated with the questions. You need 50% to have a mark of 4 and 75% to have a 5.
Author and responsible for the examination: Pierre Nugues

1 Closed Book Part: Questions

In this part, no document is allowed. It is worth 100 points.

1. **Introduction.** Cite three applications that use natural language processing to some extent. 3 points

2. **Regular expressions.** Describe what a concordance is and give all the **case-insensitive** concordances of the string *kvinna* with one word before and one word after, if any, in the text below including the title. In this definition, a concordance does not need to be a word (i.e. a string bounded by white spaces): 3 points

   > **Kvinna fastnade i maskin i Mjölby – allvarligt skadad.**
   > En arbetsplatsolycka inträffade under onsdagseftermiddagen i Mjölby. Enligt polisen har en kvinna förts till sjukhus med allvarliga skador.
   >
   > Larmet inkom 17.50 och gällde då en fastklämd person.
   >
   > Enligt räddningstjänsten ska en person ha fastnat med armen i någon form av maskin på en gård i Mjölby kommun.
   >
   > Polisen meddelar under kvällen att det handlar om en kvinna i 55-årsåldern och som förts till sjukhus med skador som bedöms som allvarliga. Kvinnan är vaken och talbar, enligt polisen.
   >
   > Polisen har upprättat en anmälan om vållande till kroppsskada genom arbetsplatsolycka.

   Source: Axel Brantemo, *SVT Nyheter*, 2025-09-10, `https://www.svt.se/nyheter/lokalt/ost/person-fastnade-i-maskin-i-mjolby-allvarligt-skadad`, retrieved September 30, 2025

3. **Regular expressions.** Identify what the regular expressions in the list below match in the text above including the title (identify all the matches

and just write one word before and after, or write no match if there is no match). Mark a matching space with the symbol: ␣: 15 points

List of **case-sensitive** regular expressions[1]:

1. `Mjölby\.`
2. `Mjölby.`
3. `^Kvinna`
4. `\p{Lu}vinna`
5. `\p{Ll}vinna`
6. `\p{N}+`
7. `(\p{N})\1`
8. `[\p{N}\p{P}]{5}`
9. `\p{Ll}{20}`
10. `\s[a-z]å[a-z]{3}\s`

4. Encoding. Describe in three to four sentences for each item what is: 8 points

1. The set of characters matched by `\p{N}` or `\p{Number}`
2. The set of characters matched by `\p{Greek}`
3. The set of characters matched by `\P{Greek}`
4. The Unicode character properties `\p{...}`
5. The Unicode character properties `\P{...}`
6. The set of characters matched by `[^\s\p{L}]`
7. The difference between `\w` and `\p{L}`
8. `'\N{GREEK SMALL LETTER GAMMA}'`

5. Regular expressions. In this exercise, you will write a small program to extract geographical coordinates from text with regular expressions. You will proceed step by step to reach this goal.

Latitudes and longitudes are coordinates used to position locations on Earth. Latitudes are north-south angles, where the origin is the equator and longitudes are east-west angles where the standard origin, the Prime meridian, is now the Greenwich meridian.

As an example, Lund's coordinates are

55° 42′ 30″ N 13° 11′ 57″ E,

where 55° 42′ 30″ N (north) is the latitude and 13° 11′ 57″ E (east), the longitude.

Before wikipedia, people used to find such coordinates in atlases, gazetteers, or encyclopedias. In the 7th edition of *Encyclopædia Britannica* from 1830 the structure of coordinates follows a regular pattern as in this example:

[1] `Ll` corresponds to lowercase letters

> RAJOORAH, a town of Hindustan, in the province of Aurungabad, forty miles south-west from Nandere. Long. 77. 15. E. Lat. 18. 38. N.

The coordinates show at the end of the entry; they are preceded by either *Lat.* or *Long.* and consist of up to three numbers representing the degrees, minutes, and seconds (DMS). These numbers may be followed by a dot. In the example above, the coordinates of Rajoorah are given in degrees and minutes (DM). The seconds (S) are missing. In some entries, we only have the degrees (D).

In the exercise, you will extract the coordinates from the text and format them so that they can serve as input to the Python `geopy` module. This module creates internal representations of latitudes and longitudes. It accepts a string input in the form of degrees, minutes, and seconds (DMS) with combinations of degrees only, degrees and minutes, and degrees, minutes, and seconds. Here are a few examples:

```
geopy.Point("52 N 7 1' E")
geopy.Point("52 10' N 7 1' E")
geopy.Point("52 10' 20\" N 7 1' 15\" E")
```

Note the order: latitudes are followed by longitudes and cardinal directions are placed after the numbers. Once your program is finished, its output for Rajoorah should be:

```
"18 38' N 77 15' E"
```

To prepare this exercise, your teacher downloaded the text of this encyclopedia from GitHub[2], where the entries are split in files, one file per encyclopedic entry. Each entry consists of a legal disclaimer followed by the entry content, here the city of Aahus in Germany. See the example below. The disclaimers have all the same pattern.

```
+======================================================================+
 ENCYCLOPEDIA BRITANNICA, SEVENTH EDITION: A MACHINE-READABLE TEXT
 TRANSCRIPTION (v3.1), The Nineteenth-Century Knowledge Project, 2024
 nckp@temple.edu, https://tu-plogan.github.io/.

 License: CC-BY-4.0, https://creativecommons.org/licenses/by/4.0/.

 Source: Encyclopaedia Britannica: A Dictionary of Arts, Sciences,
 and General Literature. 7th ed., 21 vols. Edinburgh: Adam and
 Charles Black, 1830-1842. Image scans: Natl. Library of Scotland.

 This entry: 7th edition, volume 2, page 2 [7:2:2]
+======================================================================+

AAHUS, a little town of Germany, in the circle of Westphalia and bishopric
of Munster. It is the capital of Aahus, a small district; has a good castle;
and lies northeast of Coesfeldt. Long. 7. 1. E. Lat. 52. 10. N.
```

[2]Nineteenth-Century Knowledge Project: `https://tu-plogan.github.io/`

1. Your first regular expressions will extract the text of the entry from the file and discard the legal disclaimer. The content of the file will be stored in a string called `file_content` and you will use the `regex` module.

   (a) Write a regex called `ruler_re` that matches a string consisting of a starting `+`, a sequence of `=`, and an ending `+`: 2 points

   ```
   ruler_re = ...
   ```

   This corresponds to the first line of the disclaimer;

   (b) Write a regex, `disclaimer_re`, that matches the complete disclaimer: 2 points

   ```
   disclaimer_re = ...
   ```

   It consists of starting and ending lines as above and a sequence of any characters in-between;

   (c) By default, the dot metacharacter, `.` , does not match newlines. What is the modifier that changes this behavior? 1 point

   (d) Write a regex, `file_content_re`, that matches the file content consisting of the disclaimer and the entry content, here the description of Aahus. You will remember the entry content in a backreference. 2 points

   ```
   file_content_re = ...
   ```

   (e) Finally, using appropriate Python functions, write a function

   ```
   def extract_entry(file_content: str) -> str:
       ...
       return entry
   ```

   that takes the string representing the content of a file (`file_content`) and outputs the entry text: `entry`. 2 points

   For the Aahus file, the output should be the string:

   ```
   AAHUS, a little town of Germany, in the circle of Westphalia and
   bishopric of Munster. It is the capital of Aahus, a small district;
   has a good castle; and lies northeast of Coesfeldt. Long. 7. 1. E.
   Lat. 52. 10. N.
   ```

2. Now that we have the text of the entry, you will extract the two strings containing the coordinates.

   Note that in the exercise, because of the associativity rules, you will have to group regexes using parentheses. By default, parentheses as in `(expr)` create a backreference. If you want simply a grouping operation, i.e. associate elements, use `(?:expr)` instead that will not create a backreference.

   (a) Write a regular expression, `coords_re`, that extracts a latitude or a longitude from a string. Your regex should match the strings starting with either `Lat.` or `Long.`, followed by one (D), two (DM), or three numbers (DMS), and ended by either `N` or `S`, or `E` or `W`. The numbers and the cardinal directions can be followed by an optional dot. 3 points

   ```
   coords_re = ...
   ```

   Given the Aahus text as input, your regex should match two strings: `Long. 7. 1. E.` and `Lat. 52. 10. N.`

(b) Given a string containing a longitude or a latitude, write a function that returns the list of numbers composing the coordinates and the direction: 3 points

```
def extract_coords_vals(coords: str) -> list:
    ...
    return num_dir
```

As results, you should have:

```
>>> extract_coords_vals('Lat. 47. 37. N.')
['47', '37', 'N']
>>> extract_coords_vals('Long. 23. 29. E.')
['23', '29', 'E']
```

As suggestion, you can use a regex to match the numbers and a second one the cardinal direction. Note that the abbreviations `Long` and `Lat` are redundant with the cardinal directions and will be discarded.

(c) To help you, your teacher wrote a `format_coords(coords_val: list)` function that given a list input of coordinate values returns a string:

```
>>> format_coords(['23', '29', 'E'])
"23 29' E"
```

Using it and the previous functions, write a function that extracts the latitudes and longitudes from the text of the entry and returns a string compatible with `geopy`; 3 points

```
def extract_and_format_coords(entry_text: str) -> str:
    ...
    return lat_long
```

For Aahus, the input will consist of the entry text and the output should be:

```
"52 10' N 7 1' E"
```

3. Using the `geopy` and `folium` modules, your teacher extracted 3088 coordinates from the 21,118 entries and plotted them on a map; see Figure 1. Comment the clusters briefly knowing that the 7th edition of *Encyclopædia Britannica* was published between 1830 and 1842. 1 point

6. Subtokenization. In this exercise, you will apply the byte-pair encoding (BPE) algorithm to a very small corpus:

▁meddelar▁att▁det▁handlar

1. Give the start vocabulary BPE creates from the corpus; 1 point
2. Explain in a few sentences how BPE creates the subtokens from the start vocabulary and the corpus; 2 points
3. Give the first pair BPE creates. Note that three answers are possible; 1 point
4. Give the second pair knowing the first pair; 1 point
5. Give the third pair. 1 point
6. Using these three pairs, tokenize the corpus. 1 point

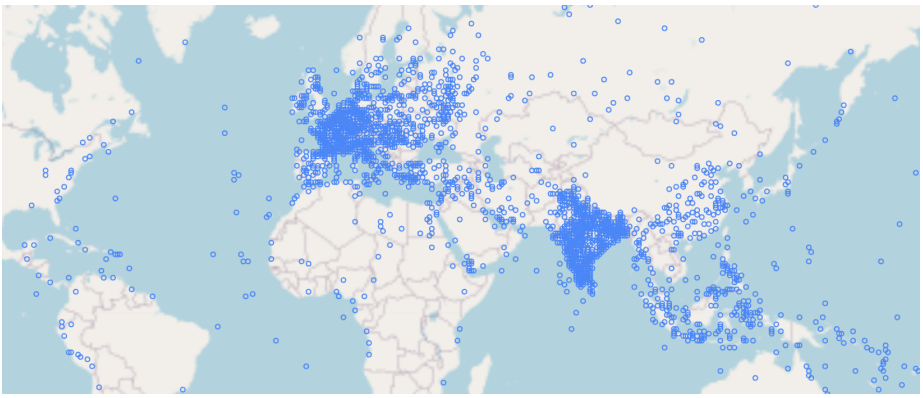

Figure 1: Locations with coordinates mentioned in the 7th edition of *Encyclopædia Britannica*

7. Language models. Rewrite the likelihood of

$$P(\text{Kvinnan är vaken och talbar})$$

using:

1. A unigram approximation; 1 point
2. A bigram approximation; 1 point
3. A trigram approximation; 1 point

8. Language models. Given a product of word probabilities:

$$\prod_{i=1}^{n} P(w_i)$$

1. Write the geometric mean of this product; 1 point
2. Write the mean negative log probability of this product (use $\log_2$). Note that this is also called negative loglikelihood or NLL; 1 point
3. Write the perplexity (use powers of 2); 1 point
4. Show this perplexity is equivalent to: 2 points

$$\frac{1}{\sqrt[n]{\prod_{i=1}^{n} P(w_i)}}.$$

9. Machine learning. Given two documents, D1 and D2, reduced to a few words:

D1: Kvinna fastnade i maskin i Mjölby
D2: maskin på en gård i Mjölby kommun

1. Vectorize the two documents using a bag-of-word encoding. 1 point

$$\begin{aligned} \mathbf{v}_{D1} &= (\ldots) \\ \mathbf{v}_{D2} &= (\ldots) \end{aligned}$$

Use the parameter index in Table 1. As parameter values in the vectors, use the word counts in each document.

	en	fastnade	gård	i	kommun	Kvinna	maskin	Mjölby	på
$\mathbf{v}_{D1}$:	0	0	0	0	0	0	0	0	0
$\mathbf{v}_{D2}$:	0	0	0	0	0	0	0	0	0

Table 1: The words in the documents. Use the same order to create your vectors

2. Compute the dot product of the two vectors: $\mathbf{v}_{D1} \cdot \mathbf{v}_{D2}$; 1 point
3. The norms of $\mathbf{v}_{D1}$ and $\mathbf{v}_{D2}$: $||\mathbf{v}_{D1}||$ and $||\mathbf{v}_{D2}||$; 1 point
4. The cosine of $\mathbf{v}_{D1}$ and $\mathbf{v}_{D2}$ (do not to reduce its value). 1 point

10. Machine learning. Your teacher wrote a feedforward network that he derived from the `nn.Module` class to detect the language of a text:

```
class FFN(nn.Module):
    def __init__(self, input_dim):
            super().__init__()
            self.fc1 = nn.Linear(input_dim, 5)
            self.relu = nn.ReLU()
            self.fc2 = nn.Linear(5, 3)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        return self.fc2(x)

model = FFN(10)
```

1. How many languages can we detect with this network? 1 point
2. Sketch a network representing this architecture; 2 points
3. Tell the mathematical operation associated with `nn.Linear`; 2 points
4. Describe what is a `torch.relu()` function (here implemented with `nn.ReLU()`); 1 point
5. We denote $\mathbf{x}$ the input vector representing a text and $\hat{\mathbf{y}}$, the output. What are the sizes of $\mathbf{x}$ and $\hat{\mathbf{y}}$; 1 point
6. Deriving a class from `nn.Module` is unnecessary complex for such an architecture. Write an equivalent model with `nn.Sequential` 2 points

   ```
   model = nn.Sequential(
     ...
   )
   ```

11. Machine learning. Your teacher created a training program with random input matrices and output vectors:

```
X = torch.randn((100, input_dim))
Y = torch.randint(0, 3, (100,))
```

1. Explain what this loss function is: 1 point

```
loss_fn = nn.CrossEntropyLoss()
```

2. Give the value of this loss for an output vector $\hat{\mathbf{y}} = (0.2, 0.1, 0.7)$ and a true value $\mathbf{y} = (0, 0, 1)$. Do not compute it. Just give the mathematical formula. 1 point
3. Your teacher used a stochastic gradient descent optimizer: 2 points

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

Explain in a few sentences what a stochastic gradient descent is.

4. Finally, your teacher wrote a simple training loop:

```
train_loss = []
for _ in range(epoch):              # 1
    loss_val, cnt = 0, 0
    for x, y in zip(X, Y):          # 2
        y_hat = model(x)            # 3
        loss = loss_fn(y_hat, y)    # 4
        optimizer.zero_grad()       # 5
        loss.backward()             # 6
        optimizer.step()            # 7
        loss_val += loss.item()
        cnt += 1
    train_loss += [loss_val/cnt]
```

Explain each numbered line in a few sentences (one point per line) 7 points

12. Machine learning. The formula of self-attention is:

$$\text{softmax}\left(\frac{XX^\intercal}{\sqrt{d_{\text{model}}}}\right)X,$$

1. Describe what X represents relatively to the input words; 2 points
2. Describe what $XX^\intercal$ represents relatively to the input words; 2 points
3. What is the purpose of the $\sqrt{d_{\text{model}}}$ division; 1 point
4. Why do we apply a softmax function? 2 points
5. Finally what is the result when we multiply softmax $\left(\frac{XX^\intercal}{\sqrt{d_{\text{model}}}}\right)$ and X. 2 points

13. Machine learning. Figure 2 shows a general scheme to use pretrained transformer encoders.

1. Describe how transformer encoders (BERT) are pretrained. 2 points
2. Describe what a transformer encoder learns when pretrained on a large corpus. 1 point
3. Describe what fine-tuning is in the context of a classification with a transformer encoder like BERT. 1 point
4. Describe what is the purpose of the head and what is the simplest head you could use with a BERT classification. 1 point

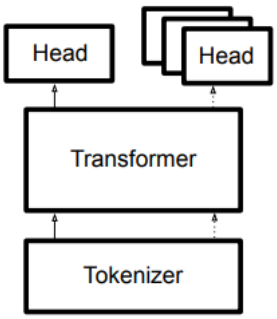

Figure 2: Picture from Wolf et al., Transformers: State-of-the-art Natural Language Processing, EMNLP Demos 2020.

2 Programming Problem

In this part, documents are allowed. It is worth 100 points.

In this programming problem, you will analyze a technique to fine-tune transformers, and more generally neural networks, with matrices of reduced sizes. You will follow the paper *LoRA: Low-Rank Adaptation of Large Language Models* by Hu et al. (2021) and write a toy implementation of it.

As programming language, you will use Python with the PyTorch module. The accent is not on knowing precisely the Python or PyTorch functions. If, at a point of the examination, you know a function exists, but you do not remember it precisely, invent a name and describe the arguments you are using.

2.1 Mathematical Warmup

We will start with a mathematical reminder on matrix multiplication. Remember that a matrix is a rectangular array of numbers and its size is the number of rows by the number of columns.

1. Given a matrix W of size (d, d), ($W \in \mathbb{R}^{d \times d}$), what is the number of parameters (elements) of W? 2 points
2. Given a matrix B of size (d, r) and a matrix A of size (r, d) ($B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times d}$), what is the number of parameters of B, resp. A? 2 points
3. What is the size of the product BA? 2 points
4. Can we add W and BA? 2 points
5. If $d = 1024$ and $r = 4$, what would be the sizes of W, B, and A and the number of parameters they have? 2 points

2.2 Overview

Given a pretrained matrix W of size (d, d), the idea of LoRA is to fine-tune it through the product of two matrices B and A of size (d, r) and (r, d), where r is much lower than d. The fine-tuning process only fits B and A and adds BA to W.

1. Read the paper abstract and the introduction (Sect. 1). Summarize them in about 5 to 10 lines. 8 points
2. In this introduction, last paragraph before the bullets, what are the examples of values the authors cite for d and r? 2 points

2.3 Architecture

1. Read Sect. 4.1 of the paper and summarize the rationale behind the LoRA technique in about 10 lines. 8 points

   Note that in the paragraph **No Additional Inference Latency**, you have this sentence:

   > Note that both W^0 and BA are in $\mathbb{R}^{d \times k}$,

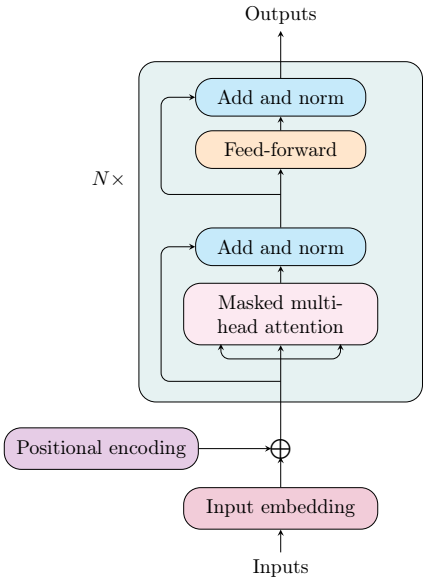

Figure 3: Transformer decoder

In this examination, you will use $k = d$. In the textbook, d is denoted d_{model}. In this article, the authors either use d or d_{model} to mean the same thing.

2. Read Sect. 4.2 of the paper. Name the matrices of a transformer that will undergo an adaptation and the matrices that will not be adapted. 4 points

3. Using decoder architecture in Fig. 3 and the multihead attention in Fig. 4, redraw quickly a decoder and mark the matrices where the adaptation takes place. Note that Fig. 4 has multiple heads. In this examination, you will assume there is only one head. (i.e. draw only one head). 6 points

2.4 GPT-2 Parameter Count

We will focus on an OpenAI model called GPT-2 medium, an ancestor of GPT-3 and chatGPT. This model uses 50,257 subtokens, has an embedding dimension, d_{model}, of 1024, and has 24 layers. In this question, you will estimate, step by step, its parameter count, excluding the biases and a few other details.

1. Compute the number of trainable parameters of the token embedding layer (Exclude the position encoding). 3 points

2. Compute the number of parameters of one W matrix in the attention layer. 2 points

3. Compute the number of parameters of the four W matrices in the attention layer 2 points

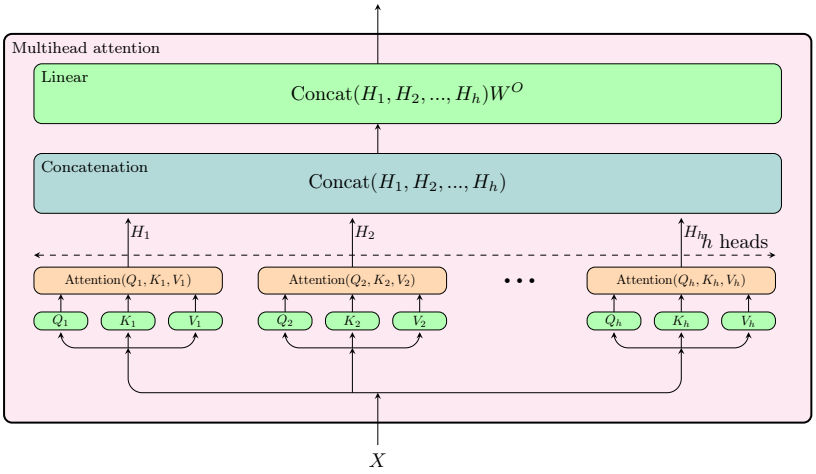

Figure 4: The multihead attention.

4. Compute the number of parameters of the feedforward component. In GPT-2, as in the original transformer, this component consists of two matrices with a hidden dimensionality $d_{ff} = 4 \cdot d_{\text{model}}$. Note that the hidden dimensionality corresponds to the output dimensionality of the first layer and the input dimensionality of the second one. 2 points

5. Show that GPT-2's number of parameters for the 24 layers is about 353.5 M and roughly corresponds to the first row in Table 3 of the paper. 2 points

2.5 LoRA Parameter Count

Read the last paragraph on page 6 entitled **LoRA**. You will now count the number of parameters required by the B and A matrices. You will use $r = 4$ and adapt W^Q and W^V only.

1. Compute the number of parameters required by A and B in one layer. 3 points

2. Show the number of parameters required by A and B for the 24 layers is about 0.39 M.[3] 3 points

2.6 Implementation

You will now implement a LoRA fine-tuning mechanism in Python. Adding it to a transformer would take more time than this examination permits. We will simplify the setup and use a feedforward network of two layers instead[4]. This architecture is equivalent to the one you implemented in the language detection laboratory.

[3]Note that there is a mistake in Table 3, last row of GPT-2. Your teacher checked the number by counting the parameters of the model in Microsoft's GitHub repository: `https://github.com/microsoft/LoRA/releases/download/GPT-2/gpt2_md_lora_e2e.pt`

[4]Should you want to read the full code described in the paper after the examination, visit `https://github.com/microsoft/LoRA`

You will use LoRA to fine-tune these two linear layers. As in Eq. 3 of the paper, we will have for each layer:

$$W\mathbf{x} + BA\mathbf{x},$$

where only B and A will be trainable in the fine-tuning step.

1. Following Fig. 1 in the paper, draw a sketch of a two-layer feedforward network with a ReLU activation in-between, where you will add a LoRA adaptation on each of these layers. 4 points

2. Create a class implementing a feedforward network, `FFN`, of two layers with an input dimensionality of `in_dim`, an output dimensionality of `out_dim`, and a hidden dimensionality between the two layers of `hidden_dim`. You will add a ReLU function between the two layers and you will derive your class from `nn.Module`. Reuse the structure below to compose your class. 6 points

```
class FFN(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim):
        self.fc1 = ...
        self.relu = ...
        self.fc2 = ...

    def forward(self, x):
        ...
        return x
```

When creating this object,

```
>>> ffn = FFN(1024, 4096, 1024)
```

You should have

```
>>> ffn
FFN(
  (fc1): Linear(in_features=1024, out_features=4096, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=4096, out_features=1024, bias=True)
)
```

3. Write a LoRA class that creates the B and A matrices. To simplify the code, B and A will be `Linear` objects with no bias and you will ignore the initialization proposed in the paper: 7 points

```
class LoRALayer(nn.Module):
    def __init__(self, in_dim, out_dim, rank):
        self.A = ...
        self.B = ...

    def forward(self, x):
        ...
        return x
```

When creating this object,

```
>>> lora = LoRALayer(1024, 4096, 4)
```

You should have

```
>>> lora
LoRALayer(
  (A): Linear(in_features=1024, out_features=4, bias=False)
  (B): Linear(in_features=4, out_features=4096, bias=False)
)
```

4. Now you will create a `LinearLoRA` class that replicates exactly the layer in Figure 1 in the paper.

   The `__init__` method will receive as input a `linear` object already created and corresponding to W on the figure. `__init__` will create a `lora` object corresponding to the A and B matrices. You will reuse your `LoRALayer` class for this. The `forward` method will return the result as in Figure 1. 8 points

   ```
   class LinearLoRA(nn.Module):
       def __init__(self, linear, rank):
           super().__init__()
           self.linear = ...
           self.lora = ...

       def forward(self, x):
           return ...
   ```

   Note that the in and out dimensions of a `Linear` object are called `in_features` and `out_features`. You access them with the name of the object followed by a dot and the name of the parameters: `linear.in_features` and `linear.out_features`.

   When creating this object,

   ```
   >>> llora = LinearLoRA(nn.Linear(1024, 4096), 4)
   ```

   You should have

   ```
   >>> llora
   LinearLoRA(
     (linear): Linear(in_features=1024, out_features=4096, bias=True)
     (lora): LoRALayer(
       (A): Linear(in_features=1024, out_features=4, bias=False)
       (B): Linear(in_features=4, out_features=4096, bias=False)
     )
   )
   ```

5. Finally you will create the network of two layers with a LoRA adaptation. You will start from the template below, where the `ffn` parameter is an instance of `FFN`. You access the layers of `ffn` with a dot like this: `ffn.fc1` and `ffn.fc2`. 8 points

```
class FFNLoRA(nn.Module):
    def __init__(self, ffn, rank):
        super().__init__()
        self.fc1 = ...
        self.relu = ...
        self.fc2 = ...

    def forward(self, x):
         ...
        return x
```

`FFNLoRA` class is quite similar to `FFN`. You should replace `Linear` with `LinearLoRA` and the good parameters.

When creating this object,

```
>>> ffnlora = FFNLoRA(ffn, 2)
```

You should have

```
>>> ffnlora
FFNLoRA(
  (fc1): LinearLoRA(
    (linear): Linear(in_features=1024, out_features=4096, bias=True)
    (lora): LoRALayer(
      (A): Linear(in_features=1024, out_features=2, bias=False)
      (B): Linear(in_features=2, out_features=4096, bias=False)
    )
  )
  (relu): ReLU()
  (fc2): LinearLoRA(
    (linear): Linear(in_features=4096, out_features=1024, bias=True)
    (lora): LoRALayer(
      (A): Linear(in_features=4096, out_features=2, bias=False)
      (B): Linear(in_features=2, out_features=1024, bias=False)
    )
  )
)
```

6. Now that we have a feedforward network of two layers with a LoRA adaptation, your teacher executed this code:

```
sum(
    p.numel() for p in ffnlora.parameters() if p.requires_grad)
Answer: ---> 8414208

for param in ffnlora.parameters():
    param.requires_grad = False
for param in ffnlora.fc1.lora.parameters():
    param.requires_grad = True
for param in ffnlora.fc2.lora.parameters():
    param.requires_grad = True
```

```
sum(
    p.numel() for p in ffnlora.parameters() if p.requires_grad)
Answer: ---> 20480
```

`ffnlora.parameters()` gives the list of tensors in `ffnlora`; `Tensor.numel()` gives the number of elements (parameters) of a tensor; and `Tensor.requires_grad` activates or deactivates its gradient, i.e. makes a tensor trainable or not.

Explain the answers of the first and last statements and guess the effect of the three loops in-between. 6 points

2.7 Application

To check the program was working, your teacher applied it to language detection with five languages: French, English, Swedish, Danish, and Italian. As input he took the relative frequency counts of the alphabet letters from A to Z, hence a 26-dimensional vector and as dataset 20,000 sentences for each language. He obtained a macro-averaged F1 score of 93.24%.

He then realized he had confused Danish with German. We can excuse him as the language codes are pretty close: dan and deu. To fix it, he substituted the Danish sentences with German ones and, as training takes long time, he used a dataset of 4000 sentences per language. He carried out a few experiments and he obtained the results in Table 2. Comment these figures. 6 points

Experiment 1 Application of the feedforward model pretrained on the initial languages including Danish to the new dataset with German substituted to Danish;

Experiment 2 Application of the feedforward model retrained from scratch on the new dataset of 4000 sentences per language;

Experiment 3 We used the feedforward model pretrained on the initial languages and we further trained it on the new dataset of 4000 sentences per language;

Experiment 4 We used the feedforward model pretrained on the initial languages and we fine-tuned it with LoRA and a rank of 2 on the new dataset of 4000 sentences per language.

Exp.	Model	# param.	F1	German
1.	Pretrained feedforward model as is	0	74.09	25
2.	Retrained from scratch on the new dataset	133925	92.71	94
3.	Fine-tuned the feedforward model from the pretained state	133925	94.14	95
4.	Fine-tuned with LoRA from the pretained state	3250	94.34	94

Table 2: Results of the four experiments. # param. is the number of trainable parameters, F1 denotes the macro F1 score, and German denotes the F1 score obtained on German

2.8 A Last Word

If you found this exercise interesting, you can try to implement it after the examination. Just follow the steps of the exam.